Short Circuit

Algorithms

Grokking Algorithms
by Aditya Y. Bhargava
Second Edition

Summary by Emiel Bos

1 Data structures

Data Structure	Time Complexity (Average)			Time Complexity (Worst)			Space Complexity (Worst)
	Access	Search	Insert / Delete[1]	Access	Search	Insert / Delete	
Array	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$
Queue	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$
Linked list	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$
Hash table	N/A	$\Theta(1)$	$\Theta(1)$	N/A	$O(n)$	$O(n)$	$O(n)$
Binary search tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
B-tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Splay tree	N/A	$\Theta(\log n)$	$\Theta(\log n)$	N/A	$O(\log n)$	$O(\log n)$	$O(n)$

1.1 Memory

Data is stored in RAM, where each byte has an address. RAM is a sequential series of bytes, and RAM stands for Random Acces Memory, because you can access any position/address in RAM in constant time. If you want to store any data, you have to ask your OS, which will *allocate* (reserve) the requested amount of contiguous space and return the address of the first byte.

1.2 Arrays

Arrays are stored contiguously in RAM.

Static arrays have a fixed length. It never needs to be moved when it runs out of space, but it is highly inflexible. Accessing the i'th element is $O(1)$ (*random access*), inserting and/or removing the last element is $O(1)$ (which is just overwriting it), inserting and/or removing anywhere else is $O(n)$, because you need to shift every element.

Dynamic arrays double in size whenever they run out of space, with the new array allocated somewhere else. All time complexities are still the same. Even pushing n elements is still only $O(n)$, even when starting with an array of length 1, because it takes $1 + 2 + 4 + 8 + ... + \frac{n}{2} + n$ operations, which is never more than $2n$.

1.3 Linked lists

A *(singly) linked list* consists of nodes, each containing a value and a pointer (address) to the next node. Linked lists do not need to be stored contiguously and never need to move. Accessing and searching is $O(n)$ due to traversal (*sequential access*), except at the start or end when maintaining a `head` or `tail` pointer. Inserting/deleting is $O(1)$, because you only have to change some pointers. Note that the time complexities are basically the inverse of an array: finding the point of insertion/deletion is $O(n)$ for linked lists and $O(1)$ for arrays, while performing the insertion/deletion is $O(1)$ for linked

lists and $O(n)$ for arrays. In other words, arrays are better for reads, linked lists are better for writes.

A *doubly linked list* has nodes which also point to the previous node. Deleting at the end is now also $O(1)$, because we have a pointer to the penultimate node (`tail.prev`). Java has `LinkedList<T>`, a doubly-linked list implementation of the `List` and `Deque` interfaces.

1.4 Stacks

A *stack* is a LIFO (last in, first out) data structure which supports pushing, popping and peeking (at the top) in constant time. Stacks can be implemented as dynamic arrays or doubly linked lists.

1.5 Queue

A *queue* is a FIFO (first in, first out) data structure which supports enqueueing to the start and dequeueing from the end in constant time. Are best implemented as linked lists.

1.6 Deque

A *double-ended queue* is like a generalization of a stack and a queue which supports $O(1)$ pushing and popping to either end. Python has `deque` in the `collections` module. Java has the `ArrayDeque` and `LinkedList` container classes that both implement interface `Deque`, which you can use as a stack with `push()`/`addFirst()` and `pop()`/`removeFirst()`/`remove()` for adding/removing on top/at the front, or as a queue with `addLast()`/`add()` (at the end) and `removeFirst()`/`pop()` (at the front).

1.7 Hash tables

A *hash function* is a function where you put in a string or any kind of sequence of bytes, and you get back a number. The function has to be consistent (same input maps to the same output). This output number can be used to deterministically index into an array by modulo'ing it by the array size. A *hash table*, *(hash) map*, *dictionary* (dicts in Python), or *associative array* is a hash function with an array. Java has `HashSet<T>`/`HashMap<T,T>`.

Collisions happen when different keys map to the same index. Collisions are avoided with a good hash function – i.e., one that distributes values in the array evenly – and a low load factor. The *load factor* of a hash table is the percentage of occupied slots in the array. Having a load factor greater than 1 means you have more items than slots in your array. Each time the load factor is above something like 0.7, the array is doubled[2] and all existing key-value pairs are moved to their new hashing location (which is dependent on array size), and this is called "rehashing the array". But collisions may still occur, and they can be handled in different ways:

- *Chaining*: instead of one key-value pair per index, store a linked list per index.
- *Open addressing*: if you want to store a key-value pair and an index is occupied, you just put the key-value pair at the next unoccupied index. When searching, you search (*probe*) until the next unoccupied index (at which point you know it's not in there, or else it would be at that spot). The downside of +1'ing is that you cluster all elements, and there are better ways, like continuously squaring and modulo'ing.

Hash tables have $O(1)$ time complexity for inserting, removing and searching[3][4]. The caveat is that you can't easily traverse it in order, which would take $O(n \log n)$. They also can't contain duplicates. Use cases are fast lookup, filtering out duplicates, and caches.

1.8 Graphs

Graphs have n nodes/vertices and m edges, where the edges may have a direction (represented with an arrow rather than a line). For directed graphs, $n \leq m^2$ (vertices can point to themselves). In undirected graphs, edges don't have a direction. In directed graphs, nodes have in-neighbours (that points toward them) and out-neighbours.

Graphs are most often represented as an *adjacency list*, which concretely is either

[2] The size of the array should actually be a prime number to further avoid collisions for math reasons, so in practice it's not quite doubling it when resizing.

[3] Whereas BSTs (aka treemaps/sets) have $O(\log n)$ for all three.

[4] Actually, $O(1)$ is the average time complexity and $O(n)$ is the worst time complexity if you have a bad hash function, but you can basically assume that $O(1)$ is simply the worst case.

- a collection of node/vertex objects, each of which has a value and a list of references to other node objects.
- a hash table with the node value as key and a list of (out-)neighbours as values (in which each element is that neighbour's key). Each node must have a unique value for this to work.
- a list of node pairs.

A *topological sort* is (one of multiple possible) valid ordered lists of a directional graph. You can use DFS or Kahn's algorithm (see Section 2.7.4) to get a topological sort (both are also useful for cycle detection).

In a *weighted graph*, each edge has an associated *weight*. One way of implementing/representing a weighted graph in code is with a nested hash table: the top-level table maps each node to a nested table, which in turn maps (only) that node's out-neighbours to their weights.

Graphs are less commonly represented as a matrix, a 2D array in which each **0** is a node and each **1** is not, and then you can move up, down, left, right (and sometimes diagonally) to each neighbouring node. So the graph is laid out like a grid, with possible holes.

1.8.1 Trees

A *tree* is a *connected* (there is path between every pair of nodes) *acyclic* (there are no cycles) graph. *Rooted trees* have exactly one node that leads to all the other nodes: the *root*, which doesn't have parents. Nodes can have children and a parent. Nodes with no children are *leaf nodes*. A valid tree has exactly $m = n - 1$ edges.

A *binary tree* is a special type of tree where nodes can have at most two children. They're highly similar to linked lists, but with different terminology and no cycles. Rather than one `next` pointer, each node has two `left` and `right` pointers, and likely also a `parent` pointer. The *height* of a binary tree is the longest path from the root node to any leaf node. The *depth* of a node is the length of the path to the root. Huffman coding (the foundation for text compression algorithms) uses binary trees.

Binary search trees In a *binary search tree* (BST), everything in a node's left/right subtree is less/greater than the node's value. A balanced BST is one where each node's left and right subtree have equal height, both $\log(n)$. Searching a balanced tree is $O(\log n)$, searching a super unbalanced tree (basically a linked list) is $O(n)$, and generally it is $O(h)$, where h is the height. Finding the minimum/maximimum is easy: just keep taking left/right turns, which is still $O(\log n)$. The main advantage of BSTs over (sorted) arrays is that adding/removing elements/nodes can also be $O(\log n)$ (whereas it is $O(n)$ for arrays). It's somewhat of a middle ground between arrays and linked lists.

Adding nodes as leaf nodes is easier than adding nodes as root or parent nodes, but does result in unbalances trees. This way is very similar to searching; we essentially search for the value to insert until we reach a null, and then create and add the node there.
Removing a node is more involved. First we need to search for it. If the node has 0 children, just remove it and set the parent pointer to null, if the node has 1 child, set the parent pointer to point to that child, bypassing the node. If the node has two children, we take the leftmost descendant (with the smallest value) from the right subtree, and replace the node with that. You can also take the rightmost descendant (with the largest value) from the left subtree.

Sets and maps can be implemented with BSTs, and they're called something like `OrderedSet`/`OrderedMap` or `TreeSet`/`TreeMap`. BST maps are called `SortedDict` in Python (you need package `sortedcontainers`), `map<T,T>` in C++, and `TreeSet<T>` /`TreeMap<T,T>` in Java. As the name implies, they're a slower implementation of sets and maps than hash tables ($\Theta(\log n)$ vs. $\Theta(1)$), but you can traverse them in order, whereas this isn't possible with hash tables.

AVL trees *AVL trees* are a type of self-balancing BST, i.e. it will rebalance whenever the tree is out of balance (with a height difference between leaf nodes of more than 1) by rotating. A *rotation* is an operation on a binary tree that changes the structure without interfering with the order of the elements. It moves one node up in the tree and one node down. Lookup, insertion, and deletion all take $O(\log n)$ time in both average and worst case. Each node stores either its height or its *balance factor*, which tells which child subtree is taller and by how much. Negative values indicate the left tree is taller and vice versa, and a 0 indicates a balanced tree. -1 or 1 are okay, because AVL trees allow a maximum difference of 1. If you have the heights of each subtree, it is easy to compute the balance factor by subtracting the right subtree's high by that of the left subtree. New nodes are added as leaf nodes (with height or balance factor 0), and all its ancestors heights/balance factors are updated. If a node's balance factor becomes -2 or 2, it is immediately rebalanced. After a rebalance, you don't need to move further up the tree, because AVL trees require one rebalancing at most.

Splay trees A different take on balanced BSTs. If you have recently looked up an item, the next lookup for that item will be faster, because when you look up a node, it will make that node the new root. More generally, recently looked up nodes get clustered to the top and become faster to look up.

B-trees B-trees are a generalized form of binary tree, where each node can have more than one key and more than two children. More specifically, a node's number of children is one greater than it's number of keys. *Seek time* is the time it takes for the hard disc's read/write head to move to the correct track, i.e., it's a physical mechanism that introduces a delay. B-trees optimize this seek time. They spend more time reading each node, but they seek less because they read more data into memory in one go. For this reason, they're a popular data structure for databases. Like BSTs, everything in a node's left/right subtree is less/greater than the node's value.

Tries A *trie*, or prefix tree, is a rooted tree data structure used for retrieval of a key in a dataset of strings. It's used for autocomplete, spellchecking, IP routing, etc. Each node in the tree has a link for each possible character (which by default are null) and a boolean indicating if the current node ends a valid word that has been added (else its just a prefix).[5] If m is the word/key length, insertion is $O(m)$ time and space, searching for the exact input word and searching any word with the given prefix is $O(m)$ time and $O(1)$ space. For insertion and search, hash tables are equally as good, but they don't support prefix search.

Segment trees A *prefix/postfix* is a subarray (contiguous) that starts at the beginning/ends at the end of an array. Prefix sums allow quickly querying the sum of a subarray: if you have the prefix sum respectively product up until each index of an array, you can query the sum/product of any subarray demarcated by `i,j` in constant time as `prefixSum[j] - prefixSum[i-1]`. However, you can't update elements. Segment trees allow for both querying subarray sums and updating elements, both in $O(\log n)$. Segment trees are balanced binary trees, where each node stores the left and right (inclusive) indices of the subarray it represents, the sum of that subarray, and pointers to its left and right children, where the left child represents range `L,M` and the right child represents `M+1,R`. The tree is built bottom-up, so the root node takes `0, arr.length-1` and recursively splits that at `M = (L+R)//2` and creates the child nodes, and after both recursions, computes its sum from the sums of its children. The base case is if `L==R`, which is a leaf node representing one element. Updating an index `i` entails recursively traversing the tree, at each node picking the left child if `i <= (L+R)//2` and the right child if `i > (L+R)//2`, and at the base case/leaf node, updating the sum and recalculating all parent sums, just as when building the tree. Searching the sum of subarray `L,R` entails recursively traversing the tree as well. At each node, if `l == L && r == R`, we return the node's sum, else if `l > (L+R)//2`, we only recurse right, if `r <= (L+R)//2`, we only recurse left, and else we need to recurse both and add the results. Often, the leaf nodes aren't even reached, but we can backtrack earlier already.

1.8.2 Forest

A *forest* is a disjoint union (basically a collection) of trees.

Union-find A *union-find*, also known as a disjoint-set or merge-find, stores a collection of disjoint (non-overlapping) sets – a partitioning of a set into disjoint subsets. They're often implemented as a forest of trees data structure that can be used on undirected graphs with disjoint connected components for counting the number of connected components and for cycle detection. The trees aren't meant to accurately represent the graph. Given the number of nodes and a list of unique, undirected edges (so `[1,2]` and `[2,1]` won't both appear), we assume each node is disjoint initially. We keep track of a parent pointer for each node (stored in a hash map, mapping node value to parent node), and the rank (height) of each tree (also stored in a hash map, with the root values as keys), which we want as small as possible for best performance. Iterating through the list of edges, for each edge (i.e. pair of nodes), we find the roots of the trees of both nodes by following the nodes' parent pointers. If the roots are the same, the nodes are already in the same tree, and since edges in the list are unique, there is a cycle. Else, we parent the root of the smallest tree (`ranks[root]`) to the root of the largest tree. If both trees have the same rank, choose one of two parentings and increase the relevant rank. While finding the roots, we can do path compression, i.e. at all levels/calls of recursion, parent the result of the subrecursion (the found root) to the input node.

1.9 Priority queues

Priority queues are like queues, except the order of popping is done on the minimum/maximum value. Lookup of this value is $O(1)$ (compared to $O(\log n)$ for BSTs), because that element is always the root. Popping it and replacing it with

[5] You can also visualize it as the nodes indicating characters, with the root node representing the empty string "", but I find it more intuitive to have the links contain the characters.

the next value is $O(\log n)$ time. Adding a value is $O(\log n)$. They're often implemented with heaps, but implementations with other data structures are possible. *Heaps* are binary trees with two properties:

- The *structure property* dictates that it is a complete binary tree, meaning only the lowest level may have holes, and those holes should be on the right (so nodes are added in breadth-first order).
- The *order property* states that the value of a node should be smaller/larger than or equal to all values in both its subtrees.

Duplicate values are allowed. Under the hood, heaps use arrays. The 0'th index is skipped, and all other nodes are in the array in breadth-first order. For every node at index i, its left child is at index $2i$, its right child at index $2i + 1$, and its parent at $\lfloor \frac{i}{2} \rfloor$.

Adding a node is done by putting it at the next proper spot, and then keep swapping it with its (current) parent while it is larger/smaller than that parent (a good term for it is "percolating" upwards). Popping the highest priority node is done by returning and removing the root node, replacing it with the last node, and then percolating that down, i.e. continually replacing it with the smallest/largest of its two children.

You can heapify an existing array in $O(n)$ time by first moving the first element to last to create space for the 0'th dummy element and then looping from the back of the array to the front, percolating each element down.[6] Because $\frac{n}{2}$ nodes don't move at all, $\frac{n}{2}$ only move one down, $\frac{n}{4}$ move at most two down, etc., you get a time series that basically approximates n. Searching a priority queue for an element is $O(n)$, but that's not their use anyways. Java has the (heap-based) `PriorityQueue<T>`, which implements `Queue` and supports `add(E e)`, `peek()` and `poll()` (which pops the head).

2 Algorithms

2.1 Big O notation

Big O notation tells you how fast an algorithm is and how the running time or memory allocation size increases as the input size increases, assuming worst case scenarios. It is expressed as the number of operations. Constants are left out, because those capture hardware/performance/implementation differences that are irrelevant and are therefore abstracted away. Common complexities are, from fastest to slowest:

- $O(1)$, or *constant time*, e.g. hash table lookup.
- $O(\log n)$, or *logarithmic time*, e.g. binary search.
- $O(n)$, or *linear time*, e.g. linear search.
- $O(n \log n)$, or *linearithmic time*, e.g. quicksort.[7]
- $O(n^2)$, or *quadratic time*, e.g. selection sort.[8]
- $O(n^3)$, or *cubic time*.
- $O(n^k)$, or *polynomial time*, where k is a constant. A generalization of quadratic and cubic time.
- $O(k^n)$, or *exponential time*.
- $O(n!)$, or *factorial time*, e.g. traveling salesman.

2.2 Recursion

Recursion happens when a function calls itself. There's no performance benefit; it is used when it makes the solution clearer. Every recursive function has a *base case* (or else it would loop infinitely) and the *recursive case*. Each of the recursive function calls takes up some memory on the *call stack*. If your recursion may go too deep and the call stack runs out of space, you can rewrite your code to use a loop instead, or use *tail recursion*.

One-branch recursion (for example: calculating a factorial) is at least $O(n)$ time and space complexity (because the function call takes up space). Two-branch recursion (for example: calculating Fibonacci sequence) is also at least $O(n)$ time and space complexity.

[6] You can actually start the algorithm at index $\lfloor \frac{i}{2} \rfloor$, because that's the last node in the array with at least one child.

[7] Exponentiation is another example of a calculation that costs $O(\log n)$, though some languages may be implemented in a way that exponentiation on integers costs $O(1)$.

[8] If you come up with an algorithm that is $O(n^2)$, always consider if sorting the input array ($O(n \log n)$) allows solving the problem in $O(n \log n)$ or better.

2.3 Dynamic programming

Dynamic programming (DP) breaks a problem down into subproblems and solves each of those while storing the results. Can replace brute-force recursion when there is a lot of repeated work being done. For a problem to be solved by dynamic programming, it needs to have:

- *Optimal substructure*: the optimal solution can be obtained by the combination of optimal solutions to its sub-problems. The subproblems are usually described by means of recursion.
- *Overlapping sub-problems*: the solutions to the subproblems are needed again and again in computing any of the superproblems. You could recompute them each time, but this often leads to exponential complexity, hence dynamic programming algorithms store[9] them for lookup later, called *memoization*.

If a problem has optimal substructure, but non-overlapping sub-problems, you'd need a divide-and-conquer algorithm instead.

Dynamic programming solutions consist of three components: a data structure to store answers to subproblems (typically a 1D or 2D array or hash map, but can also be a small, fixed number of variables), a recurrence relation describing how an answer depends on answers to subproblems, and a base case.

There are two variations, and in principle both are always possible solutions:

- Top-down DP still uses recursion, but with caching added. It starts with the main problem and works down.
- Bottom-up DP is iterative and starts with base cases and then works towards the main problem. Often called "true DP", and likely has better space complexity than top-down. In 1D dynamic programming, if the answer to subproblem i only depends on only a small constant of preceding answers $i-1, i-2, ...$, you can save space by only memorizing those last couple of answers, rotating them in a constant-size array. In some cases, the answer of a subproblem depends on all preceding answers (e.g. when you need the maximum or minimum answer), in which case you'd get $O(n^2)$ time complexity, but it is still dynamic programming.

Giveaways that a problem should be solved with dynamic programming are: asking for the minimum or maximum of something, having to make decisions that may depend on previously made decisions (very typical of a problem involving subsequences), or when you're trying to optimize something given a constraint.

2.4 Divide-and-conquer

Divide-and-conquer algorithms are two-branch recursive algorithms that divides or splits up a problem in smaller problems, each is solved recursively, until a base case is hit. Problems that can be solved with divide-and-conquer exhibit optimal substructure, but not overlapping sub-problems (see Section 2.3), so there's no point in storing solutions to sub-problems.

2.5 Binary search

Binary search of an element in a sorted array is a divide-and-conquer algorithm with $O(\log n)$ time complexity and $O(1)$ space complexity.

```
def binary_search(nums, target):
    if len(nums) == 0:
            return -1

    lo, hi = 0, len(nums) - 1
    while lo <= hi:
            mid = (lo + hi) // 2
            if nums[mid] > target:
                    hi = mid - 1
            elif nums[mid] < target:
                    lo = mid + 1
            else:
                    return mid

    return -1
```

Problems that require minimizing the maximum or maximizing the minimum often suggest a binary search approach on the range of possible values for this maximum or minimum, respectively.

[9] Usually in a 1D or 2D array, but there are plenty of problems that only require constant space.

2.6 Sorting algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$
Mergesort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$
Timsort	$\Omega(n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Tree Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(n)$
Shell Sort	$\Omega(n \log n)$	$\Theta(n(\log n)^2)$	$O(n(\log n)^2)$	$O(1)$
Bucket Sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
Counting Sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
Cubesort	$\Omega(n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$

Stable sorting algorithms preserve the original order if there's a tie with the comparison operator, while unstable sorting algorithms don't guarantee this.

2.6.1 Selection sort

The simplest sorting algorithm is *selection sort*, which goes through the list searching for the minimum element again and again. $O(n^2)$ time complexity, $O(1)$ space complexity.

2.6.2 Insertion sort

Insertion sort is stable and $O(n^2)$. Outer loop loops with `i` through the array from front + 1 to back. Everything to the left of `i` is sorted. Inner loop brings the element at `i` from `i` backwards to its place by swapping as long as it's out of order.

2.6.3 Mergesort

Mergesort is divide-and-conquer (two-branch recursion); it splits the array into two newly allocated arrays, recursively sorts each half and then merges those sorted subarrays into the original array. Time complexity is $O(n \log n)$: there are $\log n$ levels/splits that each require n operations. It's stable if you take care to prioritize the first half when merging.

```python
def mergeSort(array):
    if len(array) < 2:
        return

    left = array[:len(array)//2]
    right = array[len(array)//2:]
    mergeSort(left)
    mergeSort(right)

    i = j = 0

    # Add until we reach end of either left or right
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            array[i+j] = left[i]
            i += 1
        else:
            array[i+j] = right[j]
            j += 1

    # Add remaining elements. Note that only one of these loops will execute
    while i < len(left):
        array[i+j] = left[i]
        i += 1

    while j < len(right):
        array[i+j] = right[j]
        j += 1
```

Mergesort can theoretically be done in-place (avoiding extra array allocation), but implementing the merge step is messy and complex, and it's usually slower than the standard version because of all the extra element shifting and careful indexing.

2.6.4 Quicksort

Quicksort, developed by Tony Hoare, is similar to mergesort. It is also divide-and conquer, but no additional arrays are needed; everything happens in-place. Also, quicksort first partitions the array and then make two recursive calls, whereas mergesort first makes recursive calls for the two halves and then merges them. At each level, the rightmost element is selected as pivot, and the rest of the array is looped front-to-back, and if an element is smaller than the pivot, it is swapped with the element at a separate "swapper" pointer that also starts at the beginning and is incremented with every swap. Lastly, the pivot itself is swapped with the element at the swapper pointer, and it then separates the array in two halves, each of which is unsorted but all elements on the left side are smaller than all elements on the right side, and each side is recursed (without the pivot).

```
def quicksort(nums):
    if len(nums) < 2:
        return nums
    else:
        pivot = nums[0] # This isn't using Lomuto partition scheme, which uses the last element
        less = [i for i in nums[1:] if i <= pivot]
        greater = [i for i in nums[1:] if i > pivot]
        return quicksort(less) + [pivot] + quicksort(greater)
```

Worst time complexity is $O(n^2)$ with very unlucky pivots, but in practice it's $O(n \log n)$. Even though quicksort is $\Theta(n \log n)$ (on average) and merge sort is $O(n \log n)$ (always), in practice, quicksort's average case is faster than mergesort's average case and quicksort hits its average case way more often than its worst case. The performance of quicksort heavily depends on the choice of pivot, but always taking a random pivot will guarantee the $\Theta(n \log n)$ average runtime.

The above method of partitioning the array into halves smaller and larger than the pivot is called Lomuto partition scheme. There are many partition schemes, but this one is compact and easy to understand, though less efficient than Hoare's original scheme, e.g. when all elements are equal. With this scheme, time complexity degrades to $O(n^2)$ when the array is already in order, due to the partition being the worst possible one.

You can't sort an array in $O(n)$ time on a single CPU core, but there is a parallel version of quicksort that will sort an array in $O(n)$ time.

Quickselect, also developed by Tony Hoare, is a highly similar *selection algorithm*, which are algorithms for finding the kth smallest (or largest, by tweaking the below code slightly) element in an unordered list. The array is partitioned in two halves just like in quicksort, but because we know in which partition the desired element lies, only one side needs to be recursed. This has time complexity $\Theta(n)$ on average (which is almost always the case in practice), and $O(n^2)$ worst-case complexity, but this only happens on edge cases.

```
def kthSmallest(nums, k):
    def partition(left, right, pivot):
        nums[pivot], nums[right] = nums[right], nums[pivot]

        store_index = left
        for i in range(left, right):
            if count[nums[i]] < nums[pivot]:
                unique[store_index], unique[i] = unique[i], unique[store_index]
                store_index += 1

        nums[right], nums[store_index] = nums[store_index], nums[right]

        return store_index

    def quickselect(left, right, k) -> None:
        if left == right: # Base case; the list contains only one element
            return

        pivot = random.randint(left, right) # Select a random pivot index
        pivot = partition(left, right, pivot) # Find the pivot index in a sorted list

        if k == pivot: # If the pivot is in its final sorted position
             return
        elif k < pivot:
            quickselect(left, pivot - 1, k)
        else:
```

```
            quickselect(pivot + 1, right, k)

    n = len(nums)
    quickselect(0, n - 1, k)
    return nums[k]
```

2.6.5 Bucket sort

Bucket sort is $O(n)$, but is unstable and can only be used when all values of elements are from some small set. In one pass of the array, all possible values are counted in a separate array where the indices are the values, and in a second pass, the original array is overwritten using these counts. Only used in certain situations.

In a more general formulation, the buckets span ranges of values, and the buckets are individually sorted with some other sorting algorithm before being concatenated.

2.7 Graphs

2.7.1 Breadth-first search (BFS)

Either for determining whether a path exists between two nodes, or for finding the path along the least number of edges. BFS radiates out from the starting point and is $O(V + E)$.

```
def bfs(searched_item):
    queue = deque()
    queue += graph[searched_item]
    searched = set()
    while queue:
        current_item = queue.popleft()
            if not current_item in searched: # Avoid infinite loops (not needed for trees)
                if current_item == searched_item:
                    return True
                else:
                    queue += graph[current_item]
                    searched.add(current_item)
    return False
```

Breadth-first search cannot be done recursively, needs to be done iteratively with a queue. For a strict per-level BFS:

```
while(queue):
        for i in range(len(queue)):
                <process and add new stuff to queue>
```

2.7.2 Depth-first search (DFS)

Useful for topological sort, cycle detection, and path detection. Ill-suited for finding a shortest path.

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    if start in visited:
        return
    visited.add(start)
    for neighbor in graph.get(start, []):
        dfs(graph, neighbor, visited)
    return visited
```

DFS can also be done iteratively with a stack, in which case the algorithm is almost identical to BFS (just with a stack instead of a queue).

2.7.3 Dijkstra's Algorithm

For finding the shortest path from `start` to `end` in a weighted graph. Only works on graphs with no cycles and where all the edges are nonnegative.

1. Initialize: create two hash tables: one that maps each node to the sum of its shortest paths from `start` (initialized as ∞) and one that maps each node to its parent/preceding node on the shortest path. Set the path costs of `start`'s out-neighbours (and `start` as their parent). Create a set for processed nodes.

2. Loop until all nodes are processed:

(a) Find the node with currently the shortest path from `start`. The key idea of the algorithm is that there is no other shorter path to this node.[10]

(b) For all this node's out-neighbors of this node, if the route via this node is shorter, update that out-neighbours shortest path cost and set it's parent to this node. Add the node to the set of processed nodes.

3. Calculate the final path by following the parent nodes back to `start`.

Assuming the graph is given as a nested graph:

```
costs = {}
parents = {}
# ...initialize costs and parents here
processed = set()

node = find_lowest_cost_node(costs)
while node is not None:
    cost = costs[node]
    neighbors = graph[node] # A nested hash table
    for n in neighbors.keys():
        if costs[n] > cost + neighbors[n]:
            costs[n] = cost + neighbors[n]
            parents[n] = node
    processed.add(node)
    node = find_lowest_cost_node(costs)

# THERE IS A BETTER VERSION OF THIS THAT DOESN'T LOOP THROUGH ALL NODES EACH TIME, NAMELY WITH
↪  PRIORITY QUEUES. WILL UPDATE LATER
def find_lowest_cost_node(costs):
    lowest_cost = math.inf
    lowest_cost_node = None
    for node in costs: Goes through each node
        cost = costs[node]
        if cost < lowest_cost and node not in processed:
            lowest_cost = cost . . . sets it as the new
            lowest-cost nodelowest_cost_node = node
    return lowest_cost_node
```

2.7.4 Kahn's algorithm

Kahn algorithm is one way to get a topological sorting of a directed graph (or to determine whether it has a cycle) (DFS can also do both). Starting with an empty list for the ordered nodes and a queue of leaf nodes (with no incoming edges), you iterate until the queue is empty: pop a node from the queue, add it to the list, and for every out-neighbour: remove the edge from the graph, and add it to the queue if it has no in-neighbours anymore. If your ordered list is the total number of nodes, there are no cycles.

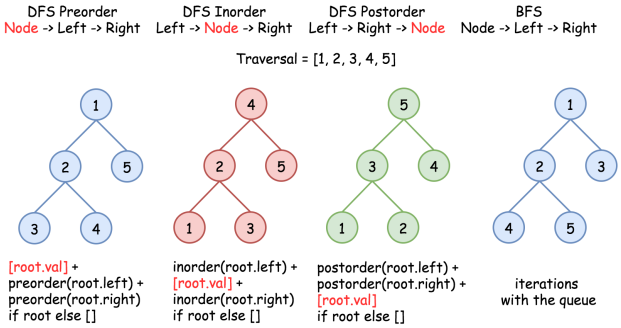
2.7.5 Trees

DFS on BST Depth-first search on BST:

```
def dfs(root):
    if not root:
        return
    dfs(root.left)
    print(root.val) # Inorder traversal
    dfs(root.right)
```

In preorder traversal, the operation on the node is done before recursing into the left subtree. In postorder traversal, the operation is done after recursing into the right subtree.

[10] This is also why negative weights break the algorithm; the algorithm relies on the fact that, once you process a node, there shouldn't be a shorter path to it, but with negative weights, a shorter path can still turn up. The Bellman–Ford algorithm can deal with negative weights.



Though recursive is almost always far easier, the three forms of traversal can also be implemented iteratively with a stack. The different forms of iterative traversal with a stack-based implementation (in order of intuitiveness):

- Preorder: at each current node, do the operation, push the right child onto the stack, set the left child as the current node, and repeat. If the current node is null, pop the current node from the stack. Repeat until the stack is empty.
- Inorder: at each current node, push that current node to the stack, set the current node to the left child, and repeat. If the current node is null, pop the current node from the stack, do the operation, and set the right node as current node. Repeat until the stack is empty.
- Postorder: at each current node, push its right child on the stack (mark as unvisited), then its left child (mark as unvisited), then itself (mark as visited), then set its left child, then pop the current node from the stack and repeat. In order to avoid pushing a right child twice, we keep track of a separate stack of booleans called visisted with the same structure and ordering as the node stack. If we pop a node from the stack that is visited, we only do the operation and don't push children, because those have been processed already.

A $O(n \log n)$ sorting algorithm for arrays is to first build a BST and then depth-first adding the values to the array like above.

The least common graph representation is the adjacency matrix, a 2D array in which each cell represents an edge from node i to node j, and the value indicates whether the edge exists. Takes $O(V^2)$ space. Uncommon on interviews.

2.8 Backtracking

Backtracking algorithms (or rather, "backtracking metaheuristics") find solutions to some computational problems, notably constraint satisfaction problems, by incrementally building candidates to the solutions, and abandoning a candidate ("backtracking") as soon as they determine that the candidate cannot possibly be (part of) a valid solution. Backtracking can be applied only for problems which can have "partial candidate solutions" and an efficient test of whether it is invalid. If that's the case, backtracking is often much faster than brute-force enumeration of all complete candidates. Conceptually, the partial candidates can be represented as nodes of a search tree. Each partial candidate is the parent of the candidates that differ from it by a single extension step; the leaves of the tree are the partial candidates that cannot be extended any further. A backtracking algorithm traverses this search tree recursively, from the root down, in depth-first order.

2.9 Sliding window algorithms

Sliding window algorithm use two pointers demarcating an interval (of fixed or variable size) and slide those across an array.
Two pointers algorithms use two pointers and generally only focus on the two elements pointed at. Sliding window algorithms are sort of a subset of two pointers algorithms, though the window in between matters.

2.10 Greedy algorithms

A *greedy algorithm* makes the locally optimal ("greedy") choice at each step, reducing each given problem into one smaller problem, i.e. it never reconsiders its choices. This is the main difference from dynamic programming, which is exhaustive and is guaranteed to find the solution. While greedy is optimal for some problems (e.g. classroom scheduling problem), for many problems, a greedy strategy does not produce an optimal solution but can heuristically yield a locally optimal solution faster (e.g. set covering problem). Greedy algorithms are a subset of *approximation algorithms*.

2.11 Decision problems

An *optimization problem* ask the minimum or maximum of something. A *decision problem* has a yes-or-no answer. An optimization problem ("What is the shortest path?") can be repurposed as a decision problem ("Is there a path of length 5?"). All problems discussed in this subsection are decision problems. The class of (decision) problems of which the solution can be found in polynomial time (and, naturally, verified in polynomial time) is called P. For some problems, there is no known way to find an answer in polynomial time, but of which answers can be verified in polynomial time; these are in class NP (standing for "nondeterministic polynomial time"). P is a subset of NP. The *P vs. NP problem* is a famous, unsolved computer science problem that asks whether every problem that is quick to verify is also quick to solve, i.e., whether P = NP.

A *reduction* happens when you reformulate a problem you can't solve into one you do know how to solve. Problem A is reducible to problem B, if an algorithm for solving problem B efficiently could also be used as a subroutine to solve problem A efficiently. If so, solving A cannot be more complex than solving B. A problem is NP-hard if any problem in NP can be reduced, in polynomial time (because else the reduction may be the bottleneck), to that problem. As a consequence, a polynomial time solution for any one NP-hard problem gives us a polynomial time solution for every problem in NP. A problem is NP-complete if it is both NP and NP-hard.

2.11.1 Satisfiability problem

The *(boolean) satisfiability problem*, often abbreviated as SAT or B-SAT, is an NP-complete problem that asks whether there exists an assignment of a given propositional logic formula's boolean variables (i.e. consistently replacing them with `true` or `false`) that makes the formula evaluate to `true`. There's no efficient algorithm; you have to fill the entire truth table (in which each row is an assignment) in $O(2^n)$ time.

SAT is the first problem that was proven to be NP-complete (the Cook–Levin theorem), meaning that all problems in the complexity class NP are at most as difficult to solve as SAT.

2.11.2 Traveling salesman problem

The *traveling salesman problem*, often abbreviated as TSP, is an NP-hard problem.

2.11.3 Set cover problem

Q: Given a set of elements $\{1, 2, ..., n\}$ (the *universe*), and a collection S of m subsets of universe whose union equals the universe, identify a smallest sub-collection of S whose union equals the universe.

A: The the set cover problem is an NP-hard problem. A greedy algorithm for polynomial time approximation chooses, at each step, the set that contains the largest number of uncovered elements

```
while uncovered_elements:
    best_subset = None
    elements_covered = set()
    for subset, elements_in_subset in stations.items():
        covered = uncovered_elements & elements_in_subset # Intersection
        if len(covered) > len(elements_covered):
            best_subset = subset
            elements_covered = covered
    uncovered_elements -= elements_covered # Set subtraction
    chosen_subsets.add(best_subset)
```

3 Problems

This sections summarizes algorithmic problems and their solutions. Sometimes, multiple solutions are possible, in which case they're ordered from good (enough) to better. For dynamic programming solutions, we generally only treat the bottom-up solution, but a top-down solution is always possible.

3.1 Knapsack

Q: Given a set of items, each with a weight and a value, determine a subset of items that maximizes the total value while satisfying the weight constraint.

A: Evaluating all subsets is $O(2^n)$. The greedy strategy of picking the most valuable thing at every step is suboptimal. 2D dynamic programming gives an optimal solution by solving the problem for smaller knapsacks and subsets of items.

You row-wise fill a table of subproblems with as columns the max weights from 1 up until the actual max weight[11], and as rows subsets of items, with each row adding another item (the order doesn't matter, but it needs to be the same order as the order of the items in the weight vector). Every cell is calculated as follows:

$$M_{i,j} = \max(M_{i-1,j}, \mathbf{v}_i + M_{i,j-\mathbf{w}_i})$$

subject to $j - \mathbf{w}_i \geq 0$, where $M_{i,j}$ is the cell at row i and column j in matrix M, and $\mathbf{w}_i$ and $\mathbf{v}_i$ are the weight and value of the ith least heavy item. In other words, we either the take the previous max value (i.e. without the item under consideration), or we take the value of the item plus the max value of the remaining space (as long as that's at least 0).

3.2 Linked List Cycle Detection

Q: Determine whether there is a cycle in a linked lists in $O(n)$ time and $O(1)$ space.

A: A two pointers algorithm. Each iteration, the fast pointer moves by two and the slow moves by one; when they meet, there's a cycle. This is known as Floyd's fast and slow pointers algorithm or Floyd's tortoise and hare algorithm.[12]

3.3 Contains Duplicate

Q: Given an integer array, determine if any value appears at least twice in the array or if every element is distinct.

A: Loop once through the array, adding each element to a hash set and checking whether it's already in there. Time: $O(n)$ Space: $O(n)$

3.4 Contains Duplicate II

Q: Given an integer array and integer k, determine if any value appears at least twice in the array and those elements are at most k indices apart, i.e. $||i - j|| \leq k$.

A: Same as the answer to Contains Duplicate, but instead we use a sliding window and delete the $i - k$'th number in the array from the hash set. Time: $O(n)$ Space: $O(\min(n, k))$

3.5 Contains Duplicate III

Q: Same as Contains Duplicate II, but instead of equal values, we're looking for two numbers that differ at most by t, i.e. $||\mathbf{x}_i - \mathbf{x}_j|| \leq t$.

A: Same as the answer to Contains Duplicate, but instead of a hash set, we use buckets with width $w = t + 1$. The bucket of any value v can be determined with $\lfloor \frac{v}{w} \rfloor$. We return `true` if, for any value, there already exists a value in the same bucket, or whether there exists a value in the two adjacent buckets with values close enough together. Because there can be at most one value per bucket (or else we can return `true`), we can use a hash map for the buckets that maps bucket ID to its singular value. Of course, we remove values outside the window. Time: $O(n)$ Space: $O(\min(n, k))$

```
def containsNearbyAlmostDuplicate(nums, k, t):
    if t < 0:
        return False
    buckets = {}
    w = t + 1  # Increment by 1 to handle the range correctly
    for i in range(len(nums)):
        bucketIndex = nums[i] // w # Getting the bucket ID
        if bucketIndex in buckets:
            return True # There's already a value in the same bucket
        if bucketIndex - 1 in buckets and abs(nums[i] - buckets[bucketIndex - 1]) < w:
            return True # There's a value in a neigbouring bucket that's close enough
        if bucketIndex + 1 in buckets and abs(nums[i] - buckets[bucketIndex + 1]) < w:
            return True
        buckets[bucketIndex] = nums[i]
        if i >= k:
            del buckets[self.getID(nums[i - k], w)]
    return False
```

[11] These can likely be integers, but if you have decimal weights, you need to have a finer column granularity, i.e. a column step size that accounts for this.

[12] The algorithm is named after Robert W. Floyd, who was credited with its invention by Donald Knuth. However, the algorithm does not appear in Floyd's published work, and this may be a misattribution.

There's also a $O(n \log n)$ answer that uses a BST as a data structure.

3.6 Top K Frequent Elements

Q: Given an integer array and an integer k, return the k most frequent elements. You may return the answer in any order.

A: Use quickselect. Use the code in Section 2.6.4, but tweak it to return the k largest elements.

3.7 Group Anagrams

Q: Given an array of lowercase strings (without numbers), group the anagrams together. You can return the answer in any order.

A: For each string, count its characters and create a character count string from those counts, e.g. `abbccc` will be represented as `#1#2#3#0#0#0...#0`. Use these character count string representations as keys in a hash map that maps them to lists of the strings with the same character counts. Time: $O(nk)$, space: $O(nk)$, where n is the length of the string array and k is the maximum length of a string in the array.

A slower solution is to sort every string and use those as keys. Time: $O(nk \log k)$, space: $O(nk)$

3.8 Product of Array Except Self

Q: Given an integer array `nums`, return an array with the value at each index `i` equal to the product of all the elements of `nums` except `nums[i]`, without using the division operation.

A: Create an array `left` and make a pass over the input array from left to right, cumulatively multiplying the input array (so `left[i] = nums[i] * left[i-1]`), and do the same from right to left in array `right`. Then, each element in the output array is `left[i-1] * right[i+1]`. Time: $O(n)$, space: $O(n)$.

A $O(1)$ space complexity optimization is to use the output array as the `left` array, and to only maintain an integer product for the right part:

```
def productExceptSelf(nums):
    result = [0] * len(nums)
    result[0] = nums[0]
    for i in range(1, len(nums)):
        result[i] = nums[i] * result[i-1]
    prod = 1
    for i in reversed(range(1, len(nums))):
        result[i] = result[i-1] * prod;
        prod *= nums[i];
    result[0] = prod;
    return result
```

3.9 Longest Consecutive Sequence

Q: Given an unsorted array of integers, return the length of the longest consecutive elements sequence in $O(n)$ time.

A: Put all numbers in a hash set (for $O(1)$ lookup), loop through all the numbers (either through the array or the set, the order doesn't matter), and for each number `num`, check if `num-1` is in the set. If it is, this number is not the start of a consecutive sequence and we skip. If it is, count the length of the consecutive sequence that starts with this number by incrementing it and checking whether this increment is in the set. Return the maximum of such a length.

```
def longestConsecutive(nums):
    longest_streak = 0
    num_set = set(nums)

    for num in num_set:
        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            longest_streak = max(longest_streak, current_streak)
```

```
    return longest_streak
```

This is $O(n)$ time and space, because each element is processed a constant number of times; the inner while loop only runs for each consecutive sequence.

3.10 Two Sum

Q: Find the (indices of) two numbers in an array that sum to zero.

A: You can do this in one loop by storing numbers in a hash set (or hash map if you want to return indices) and looking whether the current number's complement is already in the set; if so, you've find a tuple. Time $O(n)$, space $O(n)$.

3.11 3Sum

Q: Given an integer array, return all the distinct triplets (`[nums[i], nums[j], nums[k]]`, at distinct indices) that sum to zero.

A: This is the solution to Two Sum repeated for every number (though you only have to check to the right of every number as an optimization).

```
public List<List<Integer>> threeSum(int[] nums) {
    Set<List<Integer>> triplets = new HashSet<>();

    for(int i = 0; i < nums.length; i++) {
        Set<Integer> numset = new HashSet<>();
        for(int j = i+1; j < nums.length; j++) {
            if(numset.contains(-(nums[i] + nums[j]))) {
                List<Integer> list = Arrays.asList(nums[i], nums[j], -(nums[i] + nums[j]));
                Collections.sort(list);
                triplets.add(list);
            }
            numset.add(nums[j]);
        }
    }

    return new ArrayList(triplets);
}
```

3.12 Container With Most Water

Q: Given an integer array representing the heights of vertical lines, return the size of the largest rectangle you can span with any pair, i.e. $\max_{i,j}((j-i) * \min(\mathbf{x}_i, \mathbf{x}_j))$.

A: Two pointers, starting out from the outsides of the array, of which you always move the one pointing to the smallest height inward. When they're equal, it doesn't matter which one you move inward. Of course you keep track of the maximum. The intuition is that you can safely move the smallest pointer inward, because the current other pointer is at a larger height, and either all heights preceding that other pointer are shorter than and will have registered with the current pointer, or there has been a larger height before the current pointer which can only span a larger rectangle. $O(n)$ time and $O(1)$ space.

3.13 Longest Substring Without Repeating Characters

Q: Find the longest *substring* (i.e. contiguous characters) without duplicate characters in a given string.

A: A two-pointers sliding window, where the right pointer keeps moving, and max'ing the length of the current window, until it reaches a character which is already in the set of characters currently in the window, at which point the left pointer has to be increased and those characters removed until the right pointer's character is no longer in the set. Be sure to include the character at index 0 into the set during initializiation, and to take the substring length as `right - left + 1`. Time $O(n)$, space $O(n)$.

3.14 Longest Repeating Character Replacement

Q: Find the longest *substring* (i.e. contiguous characters) of the same uppercase characters in a string of uppercase characters, in which you can convert at most k characters to another character (i.e., you have k wildcards).

A: A two-pointers sliding window, where the right pointer keeps moving, and max'ing the length of the current window.

At every iteration, a hashmap of character frequencies currently in the window[13] is updated by incrementing the character at the right pointer. Also, `maxFrequency` keeps track of the maximum of any such frequency of any character in any window seen so far, and we update that as well. If `right - left + 1 - maxFrequency <= k` the window is an invalid substring, in which case the left pointer is incremented (and the corresponding frequency decremented); otherwise, the loop continues. Note that both the window size (`right - left + 1`) as well as `maxFrequency` never decreases, which doesn't matter, because we're only interested in larger windows. Time $O(n)$, space $O(n)$.

3.15 Minimum Window Substring

Q: Given two strings `s` and `t`, return the minimum window/substring (i.e. contiguous characters) of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string.

A: Actually medium. You first create a `charCounts` hash map containing the character frequencies in `t` and a `charCount` for the total number of characters. You use a sliding window, and use `charCount < 1` to check whether you currently have a valid window and `charCounts` to keep track of how many characters of each you still need for a valid window. Since characters can enter the window of which there are already sufficient duplicates in the window, character counts in `charCounts` can get negative, only increment or decrement `charCount` if the character's count in `charCounts` is nonnegative. Time $O(n)$, space $O(n)$.

3.16 Find Minimum in Rotated Sorted Array

Q: Given a sorted but rotated array (e.g., `[4,5,6,7,0,1,2]` is rotated 4 times), return the minimum element of this array.

A: An adaptation of binary search. While `nums[left] > nums[right]`, we continue searching in the half where the "breaking point" is (where the original last/max elements is adjacent to the original first/min element), after which we return the element at the left pointer. Time $O(\log n)$, space $O(1)$. Code:

```
def find_min(nums):
    p, q = 0, len(nums) - 1
    while nums[p] > nums[q]:
        h = (p + q) // 2
        if nums[h] >= nums[q]:
            p = h + 1
        else:
            q = h
    return nums[p]
```

3.17 Search in Rotated Sorted Array

Q: Given a sorted but rotated array (e.g., `[4,5,6,7,0,1,2]` is rotated 4 times), return the index of `target` (or `-1`).

A: Use the answer to the previous question to find the index of the minimum element, which is the rotation amount of the array. Then use regular binary search, except whenever you index the midpoint in the array, offset it by that rotation amount and modulo it by the array length. Time $O(\log n)$, space $O(1)$.

3.18 Reorder List

Q: Given the head of a singly linked-list ($L_0 \to L_1 \to ... \to L_{n-1} \to L_n$), reorder the list to $L_0 \to L_n \to L_1 \to L_{n-1} \to L_2 \to L_{n-2} \to ....$

A: Find the middle node of the linked list, reverse the second part of the list (i.e. starting from the middle node), then merge the two halves. Note that you need slightly different logic for an even number of nodes than for an uneven number of nodes. Time $O(n)$, space $O(1)$.

3.19 Remove Nth Node From End of List

Q: Given the head of a linked list, remove the nth-to-last node from the list and return its head, using only one pass over the list.

A: Iterate a pointer $n + 1$ times, then start iterating a second pointer (so you essentially have a sliding window of size $n+1$), and when the first pointer is at the end, you can rewire the node at the second pointer to bypass the nth node. Time $O(n)$, space $O(1)$.

[13] In Java, the map can be a simple `int[26]` indexed with `s.charAt(q) - 'A'`

3.20 Kth Smallest Element in a BST

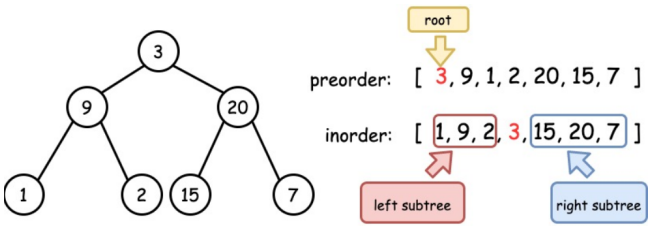
Q: Given the root of a binary search tree, and an integer k, return the kth smallest value (1-indexed) of all the values of the nodes in the tree.

A: Inorder DFS (recursive) traversal of a BST is an array sorted in ascending order.

3.21 Construct Binary Tree from Preorder and Inorder Traversal

Q: Given two integer arrays `preorder` and `inorder`, construct and return the binary tree. The node values are unique, and both traversal arrays contain the same values, just in a different order.

A: Actually hard. The key is understanding how the two arrays are structured:



The parent node is always to the left of all its children in `preorder`, so we initialize a global pointer pointing at the first position in `preorder`, which is only ever incremented. We also initialize a hash map mapping all (unique) node values to their index in `inorder`. We then recursively build the tree, maintaining left and right pointers into `inorder`. Specifically, at every node, the node value is at the pointer into `preorder`, so load that and increment the pointer. Use the hash map to find the index of the node value in `inorder`, then recurse into the left and right portions of `inorder`. You stop recursing whenever the left pointer is past the right pointer.

3.22 Binary Tree Maximum Path Sum

Q: Given the root of a binary tree, return the maximum path sum of any non-empty path. The path can be any connected sequence of nodes. Node values can be negative.

A: Actually medium. Recurse from the root. At every node you `max` a global variable containing the maximum with the node value and any path sum you'd get if it connected its childdren along the path. You return a slightly different value, namely the maximum path sum if the node is the end/beginning of a path. Time $O(n)$, space $O(n)$ (because of the call stack).

```
maxPath = -float("inf")
gainFromSubtree(root)
return maxPath

def gainFromSubtree(node): # Post order traversal. Return the max sum for a path starting at the
↪  root of subtree
    if not node:
        return 0

    gainFromLeft = max(gainFromSubtree(node.left), 0) # Use max, because if gain is negative we can
    ↪  ignore it
    gainFromRight = max(gainFromSubtree(node.right), 0) # Idem
    maxPath = max(maxPath, gainFromLeft + gainFromRight + node.val)
    return max(gainFromLeft + node.val, gainFromRight + node.val)
```

3.23 Find Median from Data Stream

Q: Implement the `MedianFinder` class, with a constructor, a method that adds the given integer to the data structure, and a method that returns the median of all elements so far, where the median is the average of the two middle elements if the number of elements is even.

A: In the constructor, initialize two priority queues, one is a max heap for the numbers smaller than the median, and the other is a min heap for the larger numbers. You add a number by comparing it to the median and putting it in the right queue ($O(\log n)$, after which you check whether one of the queues is two larger than the other, in which case you rebalance by putting the highest priority element from the larger in the smaller. Calculating the median is then simply

done by peeking and averaging the highest priority elements from both in case they're the same size, or you peek from the largest queue ($O(1)$).

3.24 Combination Sum

Q: Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations (where the order doesn't matter) of candidates where the chosen numbers sum to target. The same integers candidate may be chosen an unlimited number of times.

A: Starting at the first element with an empty list, at every index you recurse twice: once while not taking the element and moving to the next element, and once taking the element and staying at the same index. Keep track of the sum of elements in the list; and add the list to the list of lists when the current sum equals the target, and abandon/return when it is greater or equal to the target.

3.25 Word Search

Q: Given an $m \times n$ grid of characters board and a string `word`, return true if `word` exists in the grid. The word can be constructed from letters of horizontally or vertically neighbouring cells. The same letter cell may not be used more than once.

A: This really is as straightforward as your first intuition: loop through the cells, and per cell recursively search in all valid, non-visited directions, backtracking whenever the next character doesn't appear in the word.

3.26 Word Search II

Q: Same as Word Search, but instead of one word you get a list of strings words and must return all words on the board.

A: Slightly similar to the solution to Word Search, but using a trie for backtracking. Traverse the trie while recursively searching the board.

3.27 Design Add and Search Words Data Structure

Q: Design a data structure that supports `add(word)` and `search(word)`. `search(word)` returns **`true`** if there is any string in the data structure that matches `word` or **`false`** otherwise, where `word` may contain dots which can be matched with any letter.

A: Implement a trie. In `search(word)`, when you encounter a dot, you loop through all 26 subtries and recurse, returning true if any one returns true.

3.28 Number of Islands

Q: Given an $m \times n$ grid which represents a map of `1`s (land) and `0`s (water), return the number of islands, where an island is connected horizontally or vertically and is surrounded by water or the grid edge.

A: Loop through all cells in the grid, and for each unvisited `1`, call a recursive function that explores the island in all directions and marks it all as visited. The recursive function is similar to the one in Word Search. You can also simply set land from `1` to `0` to mark it as visited. Time $O(mn)$, space $O(mn)$. A BFS or Union-Find solution is also possible.

3.29 Clone Graph

Q: Given a reference of a node in a connected undirected graph, in which each node contains a unique integer value and a list of its neighbors, return a deep copy (clone) of the graph.

A: Create a hash map for storing visited nodes, in which the node's value is mapped to a reference of the cloned node. Then, clone the given node and recursively clone it's neighbors, returning a clone from the set early if the neighbor has already been copied before. Time $O(nm)$, space $O(n)$ (only the visited hash map).

3.30 Pacific Atlantic Water Flow

Q: There is an $m \times n$ rectangular island that borders both the Pacific Ocean at the top and left, and Atlantic Ocean at the right and bottom. You are given an$m \times n$ integer matrix heights. Rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from

any cell adjacent to an ocean into the ocean (let's call those "beach cells").
Return a 2D list of grid coordinates of cells from where water can flow to both the Pacific and Atlantic oceans.

A: For each ocean, you need to loop through the beach cells, and for each you start a recursive (depth-first) search, marking all cells that are equal or higher. (You can also do breath-first search with a queue; then you start by enqueueing all the beach cells.) (Note that linearly looping through the cells and checking their top and left neighbours (or right and bottom neighbours for the Atlantic part) doesn't work, since you're only considering two directions.) Then it's just a matter of looping through all cells and adding them to the list if it's reachable from both oceans. Time $O(mn)$, space $O(mn)$.

3.31 Course Schedule

Q: Given a total number of courses (the courses themselves are integer-labeled `0` through `numCourses-1`) and a 2D array of prerequisites, where each element `[a, b]` indicates that you must take course `b` first if you want to take course `a`, return a boolean indicating whether you can finish all courses.

A: First loop through the prerequisites and create an adjacency list of course dependencies; a hash map mapping course number to it's outgoing neighbours (the courses you "unlock" by taking that course). Then, run DFS on every course for cycle detection; returning **`false`** when you detect a cycle. Note that you need to maintain a recursion stack hash set specific to each DFS run (for the cycle detection), but also a hash set to store DFS-processed nodes for early returns in future DFS runs (in order to avoid quadratic time complexity). The only difference between them is that you pop nodes from the recursion stack afterwards, while you keep them in the processed set. Time $O(m+n)$, space $O(m+n)$, where m is the number of prerequisites/adjacency links and n the number of nodes/courses.
Alternatively, you can also use Kahn's algorithm (see Section 2.7.4).

3.32 Graph Valid Tree

Q: You are given an integer n – the number of nodes in an undirected graph, where each node is labeled from 0 to $n-1$ – and a 2D list of edges, where each edge `[a, b]` is undirected. Return a boolean indicating whether the edges of the given graph make up a valid tree.

A: There are multiple solutions. We'll discuss three from good to better.
You can use either DFS or BFS and check whether the visited set contains exactly n nodes. You'd also need to do cycle detection by returning **`false`** if you encounter a visited node. Because it's an undirected graph, make sure that you don't detect "trivial cycles", because edges go in two directions. Time $O(n+m)$, space $O(n+m)$, where m is the number of edges.
Alternatively, you can first do a check whether the given list of edges is exactly of size $n-1$ (a necessary property of valid trees) and then to check whether it is fully connected using any traversal algorithm. Time $O(n)$, space $O(n)$, because you start by verifying whether $m = n-1$.
Another way is union-find (see Section 1.8.2), using the optimizations of rank tracking and path compression. Time $O(n * \alpha(n))$, space $O(n)$, where α is the inverse Ackermann function and $\alpha(n)$ can in theory never go higher than 4 and in practice is 1 (so you could treat it as a small constant and just say $O(n)$). In practice, this is better than the previous two solutions, because this doesn't need to convert the input to an adjacency list (which is treated as a constant in big-O notation, so it's abstracted away).

3.33 Alien Dictionary

Q: You are given a list of strings; a dictionary of words from an alien language's dictionary, which uses the lowercase English alphabet. Assuming that the strings are sorted lexicographically by the rules of this new language, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. There can be multiple valid orderings. If the given arrangement of strings cannot correspond to any order of letters, return `""`.

A: Create a hash set `leafs` and put all characters in it. Build a directed adjacency list of character precedence (mapping chars to sets of chars) by looping through the words; for each you compare that word to the one before it and determine the first index at which the characters differ between the words. Link that index's character from the first word to that of the second, and remove the second word's character from the set of leafs. If a word is a prefix in the words preceding it, return `""`, since those two words aren't lexicographically sorted. Then, use Kahn's algorithm (see Section 2.7.4) to topologically sort the characters. If the resulting ordered string isn't equal to the number of unique characters in the words, return `""`, else return the ordered string. Time $O(c)$, space $O(a)$, where c is the total number of characters in all the words in the dictionary and a is the number of unique characters in those words. c will always be greater than a and the number of dependency relations between the characters.

3.34 House Robber

Q: Given an integer array representing amounts of money stashed in houses along a street. Return the maximum amount of money you can rob without robbing two adjacent houses. The values are nonnegative.

A: Bottom-up 1D dynamic programming. We need to only memorize the last two answers as mentioned in Section 2.3, so we don't need a full array. Loop front-to-back through the array, at every position updating `maxLastIncluded = nums[i] + maxLastExcluded` and `maxLastExcluded = Math.max(maxLastIncluded, maxLastExcluded)` (and using a temporary variable, because you're sort of swapping numbers). Time $O(n)$, space $O(1)$. A very concise Python implementation:

```
def rob(nums):
    prev, curr = 0, 0
    for n in nums:
        prev, curr = curr, max(curr, prev + n)
    return curr
```

3.35 House Robber II

Q: Same as House Robber, but the street is a circle, i.e. the last house is next to the first one.

A: `return max(rob(nums[:-1]), rob(nums[1:]))`, where `rob()` is defined above. Time $O(n)$, space $O(1)$.

3.36 House Robber III

Q: Same as House Robber, but the street is a binary tree, and you're given the `root`.

A: A pretty straightforward adaptation of the answer to House Robber:

```
def rob(root):
    def returnTupleWithMaxLootThatIncludesAndExcludesThis(node):
        if not node:
            return (0, 0)
        left = returnTupleWithMaxLootThatIncludesAndExcludesThis(node.left)
        right = returnTupleWithMaxLootThatIncludesAndExcludesThis(node.right)
        robThis = node.val + left[1] + right[1] # If we rob this node, we cannot rob its children
        dontRobThis = max(left) + max(right) # Else we can rob either of its children
        return [robThis, dontRobThis]

    return max(returnTupleWithMaxLootThatIncludesAndExcludesThis(root))
```

3.37 House Robber IV

Q: Same as House Robber, but instead of maximizing the total reward, minimize the maximum amount stolen from any single house while ensuring that at least k houses are robbed.

A: In Section 2.5, we said that *problems that require minimizing the maximum or maximizing the minimum often suggest a binary search approach on the range of possible values for this maximum or minimum, respectively*. Demarcate the search space of the possible values for the maximum loot by taking the minimum and maximum of all element values in the array. Then, use binary search to determine the minimum possible maximum loot of any burglary spree involving at least k houses. At every iteration, you take the middle of the current search space and check whether that value can be the maximum of k non-contiguous houses. This check is done by iterating through the list of house values and greedily selecting houses that have at most the value to be checked (of course skipping the next house each time we choose one). Once we rob k houses, it means the current middle value is a valid maximum, in which case we move the binary search range to the left; else we move it to the right. Time $O(n \log m)$, where m is the size of the search space, space $O(1)$.

3.38 Longest Palindromic Substring

Q: Given a string, return the longest palindromic substring (i.e. contiguous characters).

A: Solutions from good to better:

- *Bottom-up 2D dynamic programming*. You can create a $n \times n$ table of booleans, where `dp[i][j]` indicates whether the substring with inclusive bounds `i`, `j` is a palindrome. We initialize `dp[i][i] = true` for the substrings of length 1, and then `dp[i][i + 1] = (s[i] == s[i + 1])` for the substrings of length 2. Then iterate over `i`, `j` pairs with increasingly larger difference between them, setting their entry to `s[i] == s[j] && dp[i + 1][j - 1]`. Return the substring of the last such pair. Time $O(n^2)$, space $O(n^2)$.

- *Simple nested loop*. A simpler and in practice faster alternative is to simply loop over all the characters and expand from each character as the center of a potential palindrome. Also do this for the palindromes of even length. Returns the largest palindrome found. Time $O(n)$, space $O(1)$, so it's space complexity is superior than the DP solution, but in practice it's also faster, because the DP solutions always checks n^2 pairs, while this solutions abandons early.
- *Manacher's algorithm*. Out-of-scope. Time $O(n)$.

3.39 Decode Ways

Q: Given a secret message encoded as a string of numbers, in which `1` maps to `A`, `2` maps to `B`, ..., and `26` maps to `Z`, return the number of ways/different strings in which you can decode it. If the entire string cannot be decoded in any valid way, return `0`. Note that `0`s are only valid in `10`s and `20`s; else it can't be decoded.

A: Solutions:

- *Top-down 1D dynamic programming*. You create a hash map for storing strings mapped to the number of ways to decode them. At every recursion, you first check for the empty string (return `1`), whether the first character is `0` (return `0`), and whether the substring is in the map (return it). Then, recurse with the substring without the first character, and – if the first two characters are a number under 27 – add a recursion with the substring without the first two characters to it. Store and return the result. Time $O(n)$, space $O(n)$, where n is the length if the input string.
- *Bottom-up 1D dynamic programming*. Create an array with size one larger than the string, where index `i` of the array is used to store the number of decode ways for the substring from index `0` to index `i-1`. Return `0` if the first character is `0` or the string is empty. Initialize the first element as `1` and the second as `s[0] == '0' ? 0 : 1`. Then, iterate from `2` until the and of the array; the value at `i` is the sum of the element at `i-1` (if a single digit decode is possible, i.e. `s[i - 1] != '0'`) and `i-2` (if a double digit decode is possible, i.e. the first two characters make a number between 10 and 26). Return the last element of the array. Time $O(n)$, space $O(n)$, though you can have $O(1)$ space by only using the previous two elements.

3.40 Coin Change

Q: You are given an integer array representing coins of different denominations/values and an integer representing a total amount of money. Return the fewest number of coins that you need to make up that amount (assuming you have infinite of each coin type), or `-1` if it's not possible.

A: Solutions:

- *Top-down 1D dynamic programming*. Create an integer array of size the total amount plus one for the memoization, where `dp[i]` is supposed to store the minimum number of coins needed for amount `i`. Loop through all the coin types, and for each recurse with the amount minus the coin value. Return the minimum of all these recursions plus one; this is the minimum number of coins needed for the current amount. Cache and early return calculated minima. Time $O(nc)$, space $O(n)$, where n is the total amount and c is the number of coin types.
- *Bottom-up 1D dynamic programming*. Create an integer array of size the total amount plus one for the memoization, where `dp[i]` is supposed to store the minimum number of coins needed for amount `i`. Loop from 1 to the total amount, and per amount, loop through the coins from largest to smallest and store at `dp[i]` the minimum of `1 + dp[i - coin]` of these coins, or `-1` if none of the coins fit the amount or none of the coin subtractions lead to nonnegative coin amounts. Note that you can't simply take the biggest fitting coin, because that would be greedy and in this problem may not be optimal; a smaller coin may lead to a smaller number of coins. Time $O(nc)$, space $O(n)$, where n is the total amount and c is the number of coin types.
-

3.41 Maximum Subarray

Q: Given an integer array nums, find the subarray (i.e. contiguous) with the largest sum, and return that sum.

A: *Kadane's algorithm*. Loop through the array while maintaining a current sum and a maximum sum, at each element you first take the max of the current sum and 0 (because as soon as the sum is negative you can throw it away and start over), then add the current element to the sum and take the maximum of that and the maximum found so far. Return the maximum sum. Time $O(n)$, space $O(1)$.

Q follow-up: Use a divide-and-conquer algorithm to solve the problem (even though it has a worse time and space complexity).

```
max = -infinity # Set to 0 if empty subarrays are allowed
sum = 0
for x in numbers:
        sum = max(x, sum + x) # Set to max(0, sum + x) if empty subarrays are allowed
        max = max(max, sum)
return max
```

A follow-up: Write a recursive function that takes the left and right inclusive bounds. Recurse in the left and right halves and get the sums of optimal subarrays contained in both halves. Then, loop from the middle to the start, keep summing and maxing the sum with a maximum. Do the same from the middle to the end of the array, and sum the two. Return the max of the left half, the right half, and the max containing the middle.

3.42 Maximum Product Subarray

Q: Given an integer array nums, find a subarray (i.e. contiguous) with the largest product, and return that product.

A:

- *Dynamic programming*. Loop through the array while maintaining a global maximum, a current maximum, and a current minimum value. At each element, the current maximum is updated with the max of: the current element, the current maximum times the current element, or the current minimum times the current element. The current minimum is updated with the minimum of these same three values. At the end of each loop, update the global maximum by maxing it with the current maximum, and return the global maximum after the loop. In this way, 0s are handled correctly because they "reset" the current maximum and minimum, every pair of negative numbers is handled correctly because the current minimum retains the product of the elements between them, and every odd number of negatives is handled correctly because the current maximum is reset to the element after it. Time $O(n)$, space $O(1)$.
- *Emiel's algorithm*. Genius theoretical computer scientist Emiel Bos has the following interpretation of the problem. You keep a global maximum and a current product and loop through the array, maxing the current product with the global maximum. 0s break up the array as if the subsequent element were the start of the array; just reset the current product and continue. Apart from 0s, there are negative numbers. Zero or an even number of negative numbers means the maximum product subarray is the whole array; the current product will capture it's max. If there is an uneven number of negatives, then the maximum product subarray either starts after the first such negative (and ends at the array end), or ends before the last such negative (and starts at the array start). If the subarray is the latter, the current maximum product will capture it. For capturing the former, you will need to maintain a separate product that starts multiplying after the first negative number (also the first after a 0), and keep maxing that with the global maximum. Time $O(n)$, space $O(1)$.

3.43 Word Break

Q: Given a string and a dictionary of words/strings (both all lowercase English letters), return a boolean indicating whether the string can be segmented into one or more dictionary words. The same word in the dictionary may be reused multiple times in the segmentation.

A: Multiple solutions are possible, but I only bothered to write out one:

- *Top-down 1D dynamic programming*.
- *Bottom-up 1D dynamic programming*. You create a boolean array with the same size as the string length. `dp[i]` indicates whether the string up to character `i` can be made up of dictionary words. Loop through the array. At each element `i` you loop through the dictionary of words, until you find a `word` which is a suffix of the string up until character `i` and `dp[i-word.length]`, in which case `dp[i]` is true, else it's false. Return the last boolean in the array. Time $O(nmk)$, space $O(n)$, where n is the length of the string, m is the number of dictionary words, and k is the average length of dictionary words.
- *Emiel's DP adaption*. Genius theoretical computer scientist Emiel Bos proposes an adaptation of the above by putting all dictionary words in a set for constant-time lookup, determining the length l of the longest dictionary word (which is 20 in the LeetCode exercise, so it could be considered a constant), and instead of looping through all the dictionary words when finding a suffix, to only loop through the previous l characters of the string and checking the set. Time $O(m + nl)$ and space $O(n + m)$.

- *Trie implementation*.

3.44 Longest Increasing Subsequence

Q: Actually hard.[14] Given an integer array, return the length of the longest strictly increasing subsequence (i.e. not contiguous).

A: Solutions from good to better:

- *Bottom-up 1D dynamic programming.* Initialize an array with length equal to the input array and all elements equal to 1. `dp[i]` should represent the length of the longest increasing subsequence that ends with the `i`th number. Loop through the array, at every position you do a nested loop up until that position to find the element with the largest subsequence of which the corresponding number is smaller than the current one. Time $O(n^2)$, space $O(n)$.
- *Building a subsequence.* Loop through the input array while building up a strictly increasing subsequence. At each number, if it is greater than the largest number in the subsequence, add it. Otherwise, perform a linear smallest-to-largest scan through the subsequence and replace the first element that is greater than the number. This opens the door for elements that are greater than the inserted number but less than the replaced number. This algorithm does not always generate a valid subsequence of the input, but the length of the subsequence will be correct, because the length only changes when a new element is larger than the largest element in the subsequence, and this largest value will only drop when you encounter a number that is smaller only to this last number in the subsequence. Replacing earlier elements in the subsequence will increase the chance of that happening. Time $O(n^2)$, space $O(n)$.
- *Building a subsequence with binary search.* Instead of a linear scan, use binary search to find the element-to-replace in the above solution. Time $O(n \log n)$, space $O(n)$.

3.45 Unique Paths

Q: Actually easy (for the first solution; second solution is impossible). Given two integers m and n, return the number of possible unique paths from the top-left corner of a $m \times n$ grid to the bottom-right corner when you can only move right and down.

A:

- *Bottom-up 2D dynamic programming.* Initialize a two-dimensional $m \times n$ integer array with `1`s in the first row and the first column, where `dp[i][j]` indicates the number of unique paths to those coordinates, loop through all the other cells and set `d[i][j] = d[i-1][j] + d[i][j-1]`. Return `dp[m-1][n-1]`. Time $O(nm)$, space $O(nm)$.
- *Math*. There are $h + v$ moves to do from start to finish, $h = m - 1$ horizontal moves and $v = n - 1$ vertical ones. One could choose when to move to the right (to define h) and that will fix vertical ones, or one could choose when to move down (to define v) and that will fix horizontal ones.

$$\binom{h}{h+v} = \binom{v}{h+v} = \frac{(h+v)!}{h!v!} = \frac{(m+n-2)!}{(m-1)!(n-1)!} \tag{1}$$

  The best known algorithm to compute factorial function is by Peter Borwein. The idea is to express the factorial as a product of prime powers. Time $O((m+n)(\log(m+n)\log\log(m+n))^2)$, space $O(1)$.

3.46 Longest Common Substring

Q: Determine the longest common *substring* (i.e. contiguous characters) between two given strings.

A: *Bottom-up 2D dynamic programming*. Initialize a 2D array with size the letters of the two words (so you can imagine the letters of the two words along the axes of the grid). Initialize the grid with all-zeroes, then loop (row-wise or column-wise, doesn't matter) and only change the value of a cell if:

```
if word_a[i] == word_b[j]:
    cell[i][j] = cell[i-1][j-1] + 1
```

For most dynamic programming problems, the answer is in the last cell, but the solution here is the largest number in the grid, which isn't necessarily in the last cell. Time $O(nm)$, space $O(nm)$, where n and m are the length of the two respective strings.

[14] Except the first solution, which should be good enough for an interview.

3.47 Longest Common Subsequence

Q: Determine the the longest common *subsequence* (doesn't need to be contiguous) between two given strings.

A: Done similarly to longest common substring, but you set each cell as:

```
if word_a[i] == word_b[j]:
    cell[i][j] = cell[i-1][j-1] + 1 # Same as longest common substring
else:
    cell[i][j] = max(cell[i-1][j], cell[i][j-1])
```

The answer is in the last cell. Time $O(nm)$, space $O(nm)$, where n and m are the length of the two respective strings.

3.48 Jump Game

Q: Given an integer array where each element represents the maximum jump length at that position, return **`true`** if you can reach the last index from the first index, or **`false`** otherwise.

A:

- *Bottom-up 1D dynamic programming*. Initialize a boolean array of the same length, where `dp[i]` should indicate whether the end is reachable from that index. Set the last element to **`true`**. Loop back-to-front, at each element scanning the elements in that index's jump window for a **`true`**; if so, set it to **`true`**. Return `dp[0]`. Time $O(n^2)$, space $O(n)$.
- *Greedy.* Looping through the array while maintaining the maximum index you can reach so far, you only have to return **`false`** if you encounter a `0` and the maximum index is at or before this `0`'s position. Time $O(n)$, space $O(1)$.

3.49 Merge Intervals

Q: Given a 2D array of intervals (where `intervals[i] = [start, end]`), merge all overlapping intervals and return an array of the non-overlapping intervals.

A: Sort the input intervals by ascending start time. Copy them to an output array by looping through them, and if an interval's start time is less than or equal to the output array's last interval's end time (i.e. they're overlapping), merge the two and replace the last interval with the merging. Time $O(n \log n)$, space $O(\log n)$, both bottlenecked by the sorting.

3.50 Non-overlapping Intervals

Q: Given a 2D array of intervals (where `intervals[i] = [start, end]`), return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping. Intervals which only touch at a point are considered non-overlapping.

A: *Greedy.* Sort the input intervals by ascending end time. Maintain a variable for the count and a variable for the latest end time. Loop through the intervals: if an interval's start time is less than the latest end time (i.e. there's overlap), increase the counter (and don't store the interval's end time, because that's later and we want to keep intervals with earlier start times, so we're discarding it); else, store it's end time. Time $O(n \log n)$, space $O(\log n)$, both bottlenecked by the sorting.

3.51 Meeting Rooms II

Q: Given a 2D array of meeting time intervals (where `intervals[i] = [start, end]`), return the minimum number of conference rooms required.

A:

- *Priority queues*. Sort the input intervals by ascending start time. Create a priority queue with a min heap that will store end times. Loop through the intervals: add the end time of the interval to the priority queue, and pop the earliest end time in the priority queue if the start time of the current interval is later than or equal to it (intuitively, in that case a meeting room has become available for the current meeting, so the number of occupied meeting rooms doesn't increase). Return the size of the priority queue after looping. Time $O(n \log n)$, space $O(n)$.
- As a slightly different implementation of the above solution, you can also separate the start and end times into their own arrays, sort those individually, initialize two pointers at the beginning of both arrays, loop with the "start time" pointer, and increase the "end time" pointer only when if the pointed-to start time is later than or equal to

the pointer-to end time. Return the difference between the start time pointer and the end time pointer after looping. Time $O(n \log n)$, space $O(n)$.

3.52 Useful Java statements

- Sorting an array in-place: `Arrays.sort(arr)`. For more advanced datatypes and comparisons, you can supply a comparator, e.g. `Arrays.sort(strings, (sa,sb) -> Integer.compare(sa.length(), sb.length()))`.
- Getting a `String` representation of an array (e.g. for printing): `Arrays.toString(arr)`. For a multidimensional array, use `Arrays.deepToString(arr)`.
- Converting a `List<T>` to a `T[]` array: `list.toArray(new T[list.size()])`. You can even convert a `List<T[]>` to a 2D `T[][]` array: `list.toArray(new T[list.size()][])`.
- Converting a lowercase English `char` to an `int`: `c - 'a'`.
- Creating a priority queue with a min heap: **new PriorityQueue<>()**. For a max heap: `new PriorityQueue<>(Comparator .reverseOrder())`. Supports `add()`, `remove()` and `peek()`.
- Creating a double-ended queue: `new ArrayDeque<E>`. Use as a (LIFO) stack with `push()`/`addFirst()` and `pop()`/`removeFirst()`/r for adding/removing on top/at the front, or as a (FIFO) queue with `add()`/`addLast()` (at the end) and `remove()`/`removeFirst()`/pop (at the front).